

Style Guide

v1.15.x (latest)

The flexibility of Terraform's configuration language gives you many options to choose from as you write your code, structure your directories, and test your configuration. While some design decisions depend on your organization's needs or preferences, there are some common patterns that we suggest you adopt. Adopting and adhering to a style guide keeps your Terraform code legible, scalable, and maintainable.

This article discusses best practices and some considerations to keep in mind as you develop your organization's style guide. The article is split into two sections. The first section covers code style recommendations, such as formatting and resource organization. The second section covers operations and workflow recommendations, such as lifecycle management through meta-arguments, versioning, and sensitive data management.

Code style

Writing Terraform code in a consistent style makes it easier to read and maintain. The following sections discuss code style recommendations, including the following:

- Run `terraform fmt` and `terraform validate` before committing your code to version control.
- Use a linter such as [TFLint](#) to enforce your organization's own coding best practices.
- Use `#` for single and multi-line comments.
- Use nouns for resource names and do not include the resource type in the name.

- Use underscores to separate multiple words in names. Wrap the resource type and name in double quotes in your resource definition.
- Let your code build on itself: define dependent resources after the resources they reference.
- Include a type and description for every variable.
- Include a description for every output.
- Avoid overuse of variables and local values.
- Always include a default provider configuration.
- Use `count` and `for_each` sparingly.

Code formatting

The Terraform parser allows you some flexibility in how you lay out the elements in your configuration files, but the Terraform language also has some idiomatic style conventions which we recommend users always follow for consistency between files and modules written by different teams.

- Indent two spaces for each nesting level
- When multiple arguments with single-line values appear on consecutive lines at the same nesting level, align their equals signs:

```
ami           = "abc123"  
instance_type = "t2.micro"
```

- When both arguments and blocks appear together inside a block body, place all of the arguments together at the top and then place nested blocks below them. Use one blank line to separate the arguments from the blocks.
- Use empty lines to separate logical groups of arguments within a block.
- For blocks that contain both arguments and "meta-arguments" (as defined by the Terraform language semantics), list meta-arguments first and separate them from other

arguments with one blank line. Place meta-argument blocks last and separate them from other blocks with one blank line. Refer to [dynamic resource count](#) for more information on meta-arguments.

```
resource "aws_instance" "example" {  
  # meta-argument first  
  count = 2  
  
  ami          = "abc123"  
  instance_type = "t2.micro"  
  
  network_interface {  
    # ...  
  }  
  
  # meta-argument block last  
  lifecycle {  
    create_before_destroy = true  
  }  
}
```

- Top-level blocks should always be separated from one another by one blank line. Nested blocks should also be separated by blank lines, except when grouping together related blocks of the same type (like multiple `provisioner` blocks in a resource).
- Avoid grouping multiple blocks of the same type with other blocks of a different type, unless the block types are defined by semantics to form a family. (For example: `root_block_device`, `ebs_block_device` and `ephemeral_block_device` on `aws_instance` form a family of block types describing AWS block devices, and can therefore be grouped together and mixed.)

The `terraform fmt` command formats your Terraform configuration to a subset of the above recommendations. By default, the `terraform fmt` command will only modify your Terraform code in the directory that you execute it in, but you can include the `-recursive` flag to modify code in all subdirectories as well.

We recommend that you run `terraform fmt` before each commit to version control. You can use mechanisms such as [Git pre-commit hooks](#) to automatically run this command each time you commit your code.

If you use Microsoft VS Code, use the [Terraform VS Code extension](#) to enable features such as syntax highlighting and validation, automatic code formatting, and integration with HCP

Terraform. If your development environment or text editor supports the [Language Server Protocol](#), you can use the [Terraform Language Server](#) to access most of the VS Code extension features.

Code validation

The `terraform validate` command checks that your configuration is syntactically valid and internally consistent. The `validate` command does not check if argument values are valid for a specific provider, but it will verify that they are the correct type. It does not evaluate any existing state.

The `terraform validate` command is safe to run automatically and frequently. You can configure your text editor to run this command as a post-save check, define it as a pre-commit hook in a Git repository, or run it as a step in a CI/CD pipeline.

For more information, refer to the [Terraform `validate` documentation](#).

File names

We recommend the following file naming conventions:

- A `backend.tf` file that contains your [backend configuration](#). You can define multiple `terraform` blocks in your configuration to separate your backend configuration from your Terraform and provider versioning configuration.
- A `main.tf` file that contains all resource and data source blocks.
- A `outputs.tf` file that contains all output blocks in alphabetical order.
- A `providers.tf` file that contains all `provider` blocks and configuration.
- A `terraform.tf` file that contains a single `terraform` block which defines your `required_version` and `required_providers`.
- A `variables.tf` file that contains all variable blocks in alphabetical order.
- A `locals.tf` file that contains local values. Refer to [local values](#) for more information.

- A `override.tf` file that contains override definitions for your configuration. Terraform loads this and all files ending with `_override.tf` last. Use them sparingly and add comments to the original resource definitions, as these overrides make your code harder to reason about. Refer to the [override files](#) documentation for more information.

As your codebase grows, limiting it to just these files can become difficult to maintain. If your code becomes hard to navigate due to its size, we recommend that you organize resources and data sources in separate files by logical groups. For example, if your web application requires networking, storage, and compute resources, you might create the following files:

- A `network.tf` file that contains your VPC, subnets, load balancers, and all other networking resources.
- A `storage.tf` file that contains your object storage and related permissions configuration.
- A `compute.tf` file that contains your compute instances.

No matter how you decide to split your code, it should be immediately clear where a maintainer can find a specific resource or data source definition.

As your configuration grows, you may need to separate it into multiple state files. The HashiCorp Well-Architected Framework provides more guidance about [configuration structure and scope](#).

Linting and static code analysis

Terraform does not have a built-in linter, but many organizations rely on a third-party linting tool such as [TFLint](#) to enforce code standards. A linter uses static code analysis to compare your Terraform code against a set of rules. Most linters ship with a default set of rules, but also let you write your own.

Comments

Write your code so it is easy to understand. Only when necessary, use comments to clarify complexity for other maintainers.

Use `#` for both single- and multi-line comments. The `//` and `/* */` comment syntaxes are not considered idiomatic, but Terraform supports them to remain backward-compatible with earlier versions of HCL.

```
# Each tunnel is responsible for encrypting and decrypting traffic exiting
# and leaving its associated gateway.
resource "google_compute_vpn_tunnel" "tunnel1" {
  ## ...
```

Resource naming

Every resource within a configuration must have a unique name. For consistency and readability, use a descriptive noun and separate words with underscores. Do not include the resource type in the resource identifier since the resource address already includes it. Wrap the resource type and name in double quotes.

✗ Bad:

```
resource aws_instance webAPI-aws-instance {...}
```

✓ Good:

```
resource "aws_instance" "web_api" {...}
```

Resource order

The order of the resources and data sources in your code does not affect how Terraform builds them, so organize your resources for readability. Terraform determines the creation order based on cross-resource dependencies.

How you order your resources largely depends on the size and complexity of your code, but we recommend defining data sources alongside the resources that reference them. For readability, your Terraform code should “build on itself” — you should define a data source before the resource that references it.

The following example defines an `aws_instance` that relies on two data sources, `aws_ami` and `aws_availability_zones`. For readability and continuity, it defines the data sources before the `aws_instance` resource.

```
data "aws_ami" "web" {  
  ##...  
}  
  
data "aws_availability_zones" "available" {  
  ##...  
}  
  
resource "aws_instance" "web" {  
  ami                = data.aws_ami.web.id  
  availability_zone   = data.aws_availability_zones.available.names[0]  
  ##...  
}
```

We recommend following a consistent order for resource parameters:

1. If present, The `count` or `for_each` meta-argument.
2. Resource-specific non-block parameters.
3. Resource-specific block parameters.
4. If required, a `lifecycle` block.
5. If required, the `depends_on` parameter.

Variables

While variables make your modules more flexible, overusing variables can make code difficult to understand. When deciding whether to expose a variable for a resource setting, consider whether that parameter will change between deployments.

- Define a `type` and a `description` for every variable.
- If the variable is optional, define a reasonable `default`.
- For sensitive variables, such as passwords and private keys, set the `sensitive` parameter to `true`. Remember that Terraform will still store this value in plain text in its state, but it will not display it when you run `terraform plan` or `terraform apply`. Refer to [secrets management](#) for more information on how to securely handle sensitive values.
- Use [input variable validation](#) to create additional rules for your variable values in addition to Terraform's type validation. Only use variable validation when your variable values have uniquely restrictive requirements. For example, if your Terraform configuration requires two web instances, add a `validation` block to enforce it:

```
variable "web_instance_count" {  
  type          = number  
  description = "Number of web instances to deploy. This application requires at least two web instances."  
  
  validation {  
    condition     = var.web_instance_count > 1  
    error_message = "This application requires at least two web instances."  
  }  
}
```

We recommend following a consistent order for variable parameters:

1. Type
2. Description
3. Default (optional)
4. Sensitive (optional)

5. [Validation blocks](#)

Outputs

Output values let you expose data about your infrastructure on the command line and make it easy to reference in other Terraform configurations. Like you would for variables, provide a `type` and `description` for each output.

We recommend that you use the following order for your output parameters:

1. Type
2. Description
3. Value
4. Sensitive (optional)

Every variable and output requires a unique name. For consistency and readability, we recommend that you use a descriptive noun and separate words with underscores.

```
variable "db_disk_size" {  
  type      = number  
  description = "Disk size for the API database"  
  default   = 100  
}  
  
variable "db_password" {  
  type      = string  
  description = "Database password"  
  sensitive  = true  
}  
  
output "web_public_ip" {  
  type      = string  
  description = "Public IP of the web instance"  
  value      = aws_instance.web.public_ip  
}
```

Local values

Local values let you reference an [expression](#) or value multiple times. Use local values sparingly, as overuse can make your code harder to understand.

For example, you can use a local value to create a suffix for the region and environment (for example, `development` or `test`), and append it to multiple resources.

```
locals {  
  name_suffix = "${var.region}-${var.environment}"  
}  
  
resource "aws_instance" "web" {  
  ami           = data.aws_ami.ubuntu.id  
  instance_type = "t3.micro"  
  
  tags = {  
    Name = "web-${local.name_suffix}"  
  }  
}
```

Define local values in one of two places:

- If you reference the local value in multiple files, define it in a file named `locals.tf`.
- If the local value is specific to a file, define it at the top of that file.

As for other Terraform objects, use descriptive nouns for local value names and underscores to separate multiple words.

For more information, refer to the [locals block](#) and the [Simplify Terraform configuration with locals](#) tutorial.

Provider aliasing

Provider aliasing lets you define multiple `provider` blocks for the same Terraform provider. Potential use cases for aliases include provisioning resources in multiple regions within a single configuration. The `provider` meta-argument for resources and the `providers` meta-argument for modules specifies which provider to use.

providers.tf

```
provider "aws" {  
  region = "us-east-1"  
}  
  
provider "aws" {  
  
  region = "us-west-2"  
}
```

main.tf

```
resource "aws_instance" "example" {  
  
  # ...  
}  
  
module "aws_vpc" {  
  source = "./aws_vpc"  
  
}
```

- Any provider block that does not define the `alias` parameter is the default provider configuration.
- Always include a default provider configuration and define all of your providers in the same file.
- If you define multiple instances of a provider, define the default first.
- For non-default providers, define the `alias` as the first parameter of the `provider` block.

Dynamic resource count

The `for_each` and `count` meta-arguments let you create multiple resources from a single `resource` block depending on run-time conditions. You can use these meta-arguments to make your code flexible and reduce duplicate resource blocks. If the resources are almost

identical, use `count`. If some of the arguments need distinct values that you cannot derive from an integer, use `for_each`.

The `for_each` meta-argument accepts a `map` or `set` value, and Terraform will create an instance of that resource for each element in the value you provide. In the following example, Terraform creates an `aws_instance` for each of the strings defined in the `web_instances` variable: "ui", "api", "db" and "metrics". The example uses `each.key` to give each instance a unique name. The `web_private_ips` output uses a [for expression](#) to create a map of instance names and their private IP addresses, while the `web_ui_public_ip` output addresses the instance with the key "ui" directly.

```
variable "web_instances" {
  type          = list(string)
  description   = "A list of instances for the web application"
  default = [
    "ui",
    "api",
    "db",
    "metrics"
  ]
}

resource "aws_instance" "web" {
  for_each = toset(var.web_instances)
  ami      = data.aws_ami.webapp.id
  instance_type = "t3.micro"
  tags = {
    Name = "web_${each.key}"
  }
}

output "web_private_ips" {
  description = "Private IPs of the web instances"
  value = {
    for k, v in aws_instance.web : k => v.private_ip
  }
}

output "web_ui_public_ip" {
  description = "Public IP of the web UI instance"
  value      = aws_instance.web["ui"].public_ip
}
```

The above example will create the following output:

```
web_private_ips = {  
  "api" = "172.31.25.29"  
  "db"  = "172.31.18.33"  
  "metrics" = "172.31.26.112"  
  "ui"  = "172.31.20.142"  
}  
web_ui_public_ip = "18.216.208.182"
```

Refer to the [for each meta-argument documentation](#) for more examples.

The `count` meta-argument lets you create multiple instances of a resource from a single resource block. Refer to the [count meta-argument documentation](#) for examples.

A common practice to conditionally create resources is to use the `count` meta-argument with a [conditional expression](#). In the following example, Terraform will only create the

`aws_instance` if `var.enable_metrics` is `true`.

```
variable "enable_metrics" {  
  description = "True if the metrics server should be deployed"  
  type        = bool  
  default     = true  
}  
  
resource "aws_instance" "web" {  
  
  ami          = data.aws_ami.webapp.id  
  instance_type = "t3.micro"  
  ##...  
}
```

Meta-arguments simplify your code but add complexity, so use them in moderation. If the effect of the meta-argument is not immediately obvious, use a comment for clarification.

To learn more about these meta-arguments, refer to the [for_each](#) and [count](#) documentation.

.gitignore

Define a `.gitignore` file for your repository to exclude files that you should not publish to version control, such as your state file.

Do not commit:

- Your `terraform.tfstate` state file, including `terraform.tfstate.*` backup state files. The state file can contain secrets and other sensitive information. Also, version control systems do not support locking the state file so that it can only be modified by one instance of Terraform at a time.
- Your `.terraform.tfstate.lock.info` file. Terraform creates and deletes this file automatically when you run a `terraform apply` command and contains info about your [state lock](#)
- Your `.terraform` directory, where Terraform downloads providers and child modules.
- [Saved plan files](#) that you create when you include the `-out` flag when you run `terraform plan`.
- Any `.tfvars` files that contain sensitive information.

Always commit:

- All Terraform code files
- Your `.terraform.lock.hcl` [dependency lock file](#)
- A `.gitignore` file that excludes the files listed below
- A `README.md` to describe the code, input variables, and outputs

For an example, refer to [GitHub's Terraform .gitignore file](#).

Workflow style

This section reviews standards that enable predictable and secure Terraform workflows, such as:

- Pin your Terraform, provider, and module versions.
- Name your module repositories using this three-part name `terraform-<PROVIDER>-<NAME>` when using the HCP Terraform registry.

- Store local modules at `./modules/<module_name>`.
- Use the `tfe_outputs` data source or provider-specific data sources to share state between two state files.
- Protect credentials by using [dynamic provider credentials](#) or a secrets manager such as HashiCorp Vault.
- Write [tests](#) for your modules.
- Use [policy enforcement](#) on HCP Terraform to set guardrails for infrastructure operations.

Version pinning

To prevent provider and module upgrades from introducing unintentional changes to your infrastructure, use version pinning.

Specify provider versions using the [required_providers block](#). Terraform [version constraints](#) support a range of accepted versions.

Pin modules to a specific major and minor version as shown in the example below to ensure stability. You can use looser restrictions if you are certain that the module does not introduce breaking changes outside of major version updates.

We also recommend that you set a minimum required version of the Terraform binary using the `required_version` in your `terraform` block. This requires all operators to use a Terraform version that has all of your configuration's required features.

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
    }  
  }  
}
```

The above example pins the version of the `hashicorp/aws` provider to version `5.34.0`, and requires that operators use Terraform `1.7` or newer.

For modules sourced from a registry, use the `version` parameter in the `module` block to pin the version. For local modules, Terraform ignores the `version` parameter.

```
module "vault_starter" {  
  source = "hashicorp/vault-starter/aws"  
  version = "1.0.0"  
  ##...  
}
```

Module repository names

The Terraform registry requires that repositories match a naming convention for all modules that you publish to the registry. Module repositories must use this three-part name `terraform-<PROVIDER>-<NAME>`, where `<NAME>` reflects the type of infrastructure the module manages and `<PROVIDER>` is the main provider the module uses. The `<NAME>` segment can contain additional hyphens, for example, `terraform-google-vault` or `terraform-aws-ec2-instance`.

Module structure

Terraform modules define self-contained, reusable pieces of infrastructure-as-code.

Use modules to group together logically related resources that you need to provision together. For example:

- A networking module that defines a VPC, along with its subnets, gateway, and security groups.
- An application module defining all resources required for each deployment. This stack could include web servers, databases, storage, and supported networking.

Review the [module creation recommended pattern documentation](#) and [standard module structure](#) for guidance on how to structure your modules.

Local modules

Local modules are sourced from local disk rather than a remote module registry. We recommend publishing your modules to a module registry, such as the [HCP Terraform private registry](#), to easily version, share, and reuse modules across your organization. If you cannot use a module registry, using local modules can simplify maintaining and updating your code.

We recommend that you define child modules in the `./modules/<module_name>` directory.

Repository structure

How you structure your modules and Terraform configuration in version control significantly impacts versioning and operations. We recommend that you store your actual infrastructure configuration separately from your module code.

Store each module in an individual repository. This lets you independently version each module and makes it easier to publish your modules in the private Terraform registry.

Organize your infrastructure configuration in repositories that group together logically-related resources. For example, a single repository for a web application that requires compute, networking, and database resources. By separating your resources into groups, you limit the number of resources that may be impacted by failures for any operation.

Another approach is to group all modules and infrastructure configuration into a single monolithic repository, or monorepo. For example, a monorepo may define a collection of local modules for each component of the infrastructure stack, and deploy them in the root module.

```
.
├── modules
│   ├── function
│   │   ├── main.tf      # contains aws_iam_role, aws_lambda_function
│   │   ├── outputs.tf
│   │   └── variables.tf
│   ├── queue
│   │   ├── main.tf      # contains aws_sqs_queue
│   │   ├── outputs.tf
│   │   └── variables.tf
│   └── vpc
│       ├── main.tf      # contains aws_vpc, aws_subnet
│       ├── outputs.tf
│       └── variables.tf
├── main.tf
├── outputs.tf
└── variables.tf
```

The advantage of monolithic repositories is having a single source of truth that tracks every infrastructure change. However, monolithic repositories can complicate your CI/CD automation: since any code change triggers a deployment that operates on your entire repository, your workflow must target only the modified directories. You also lose the granular access control, since anyone with repository access can modify any file in it.

If your organization requires a monolithic approach, HCP Terraform and Terraform Enterprise let you scope a workspace to a specific directory in a repository, simplifying your workflows.

Branching strategy

To collaborate on your Terraform code, we recommend using the [GitHub flow](#). This approach uses short-lived branches to help your team quickly review, test, and merge changes to your code. To make changes to your code, you would:

1. Create a new branch from your main branch
2. Write, commit, and push your changes to the new branch
3. Create a pull request

4. Review the changes with your team
5. Merge the pull request
6. Delete the branch

HCP Terraform and Terraform Enterprise can run [speculative plans for pull requests](#). These speculative plans run automatically when you create or update a pull request, and you can use them to see the effect that your changes will have on your infrastructure before you merge them to your main branch. When you merge your pull request, HCP Terraform will start a new run to apply these changes.

Multiple environments

We recommend that your repository's `main` branch be the source of truth for all environments. For HCP Terraform and Terraform Enterprise users, we recommend that you use separate workspaces for each environment. For larger codebases, we recommend that you split your resources across multiple workspaces to prevent large state files and limit unintended consequences from changes. For example, you could structure your code as follows:

```
.
├── compute
│   ├── main.tf
│   ├── outputs.tf
│   └── variables.tf
├── database
│   ├── main.tf
│   ├── outputs.tf
│   └── variables.tf
└── networking
    ├── main.tf
    ├── outputs.tf
    └── variables.tf
```

In this scenario, you would create three workspaces per environment. For example, your production environment would have a `prod-compute`, `prod-database`, and `prod-networking` workspace. Read more about [Terraform workspace and project best practices](#).

If you do not use HCP Terraform or Terraform Enterprise, we recommend that you use modules to encapsulate your configuration, and use a directory for each environment so that each one has a separate state file. The configuration in each of these directories would call the local modules, each with parameters specific to their environment. This also lets you maintain separate variable and backend configurations for each environment.

```
├── modules
│   ├── compute
│   │   └── main.tf
│   ├── database
│   │   └── main.tf
│   └── network
│       └── main.tf
├── dev
│   ├── backend.tf
│   ├── main.tf
│   └── variables.tf
├── prod
│   ├── backend.tf
│   ├── main.tf
│   └── variables.tf
└── staging
    ├── backend.tf
    ├── main.tf
    └── variables.tf
```

State sharing

Since your state contains sensitive information, avoid sharing full state files when possible.

If you use HCP Terraform or Terraform Enterprise and need to reference resources across workspaces, use the `tfe_outputs` data source.

If you do not use HCP Terraform or Terraform Enterprise but still need to reference data about other infrastructure resources, use data sources to query the provider. For example, you can use the `aws_instance` data source to look up an AWS EC2 instance by its ID or tags.

Secrets management

If you do not configure remote state storage, the Terraform CLI stores the entire state in plaintext on the local disk. State can include sensitive data, such as passwords and private keys. HCP Terraform and Terraform Enterprise provide state encryption through HashiCorp Vault.

If you use HCP Terraform or Terraform Enterprise, we recommend the following:

- When using Terraform Enterprise, define and enforce a Sentinel policy to prevent use of the `local_exec` provisioner or external data sources.
- When using HCP Terraform or Terraform Enterprise, use [dynamic provider credentials](#) to avoid using long-lived static credentials.

If you use Terraform Community Edition, we recommend the following:

- Configure provider credentials using provider-specific environment variables.
- Access secrets from a secrets management system such as HashiCorp Vault with the [Terraform Vault provider](#). Be aware that Terraform will still write these values in plaintext to your state file.

If you use a custom CI/CD pipeline, review your CI/CD tool's best practices for managing sensitive values. Most tools let you access sensitive values as environment variables. For more information, refer to your CI/CD documentation.

- [Using secrets in GitHub Actions](#)
- [Gitlab pipeline security](#)
- [Integrate Vault into your CI/CD pipeline](#)

Integration and unit testing

Terraform tests let you validate your modules and catch breaking changes. We recommend that you write tests for your Terraform modules and run them just as you run your tests for

your application code, such as pre-merge check in your pull requests or as a prerequisite step in your automated CI/CD pipeline.

Tests differ from validation methods such as variable validation, preconditions, postconditions, and check blocks. These features focus on verifying the infrastructure deployed by your code, while tests validate the behavior and logic of your code itself. For more information, refer to the [Terraform test documentation](#) and the [Write Terraform tests tutorial](#).

Policy

Policies are rules that HCP Terraform enforces on Terraform runs. You can use policies to validate that the Terraform plan complies with your organization's best practices. For example, you can write policies that:

- Limit the size of a web instance
- Check for required resource tags
- Block deployments on Fridays
- Enforce security configuration and cost management

We recommend that you store policies in a separate VCS repository from your Terraform code.

For more information, refer to the [policy enforcement documentation](#), as well as the [enforce policy with Sentinel](#) and [detect infrastructure drift and enforce policies](#) tutorials.

Next steps

This article introduces some considerations to keep in mind as you standardize your organization's Terraform style guidelines. Enforcing a standard way of writing and organizing your Terraform code across your organization ensures that it is readable, maintainable, and shareable.

To learn more Terraform adoption best practices, refer to [Phases of Terraform adoption](#).

[Edit this page on GitHub](#)



Theme

System

- [Certifications](#)
- [System Status](#)
- [Cookie Manager](#)
- [Terms of Use](#)
- [Security](#)
- [Privacy](#)
- [Trademark Policy](#)
- [Trade Controls](#)
- [Accessibility](#)
- [Give Feedback](#)